

# SDS Semiconductor Device Simulator

## Semiconductor Device Simulator

### Operation Manual

Version: 1.0

Copyright: Chen Qiusong

## Table of Contents

|                                                                                                                        |    |
|------------------------------------------------------------------------------------------------------------------------|----|
| 1. Objectives and Background .....                                                                                     | 3  |
| 2. Software Modules .....                                                                                              | 4  |
| 1) Create, delete, and check DEVSIM models .....                                                                       | 4  |
| 2) Create silicon semiconductor physical relationship models .....                                                     | 4  |
| 3) Create defect state physical relationship models .....                                                              | 4  |
| 4) Create ferroelectric material relationship models .....                                                             | 4  |
| 5) Create Schottky contact relationship models .....                                                                   | 4  |
| 6) Execute device voltage sweeps .....                                                                                 | 4  |
| 7) Create one-dimensional and two-dimensional diode simulation devices .....                                           | 4  |
| 3. Installation Steps .....                                                                                            | 5  |
| 4. Module and Function Reference .....                                                                                 | 7  |
| 1) QModel_create.py .....                                                                                              | 7  |
| 2) QSimple_physics.py .....                                                                                            | 15 |
| 3) QExtraCharge.py .....                                                                                               | 35 |
| 4) QSFerro.py .....                                                                                                    | 38 |
| 5) QSSchottkyContact.py .....                                                                                          | 43 |
| 6) QSPlotSweep.py .....                                                                                                | 45 |
| 7) QSDiode_common.py .....                                                                                             | 52 |
| 5. Application Example .....                                                                                           | 55 |
| 1) mos_BotContact.geo .....                                                                                            | 55 |
| 2) mos_BotContact-intLay2.msh .....                                                                                    | 55 |
| 3) create_mos.py, the Python file for creating the device structure .....                                              | 55 |
| 4) ferro_BottomContact.py, the file for setting the device parameters and<br>simulating the device voltage sweep ..... | 58 |



## 1. Objectives and Background

This software is designed to use numerical methods to simulate electronic devices with physical properties such as semiconductors and ferroelectrics, and to use the simulation results to analyze the physical processes inside the devices, including current-voltage relationships, the generation and recombination processes of semiconductor defect states, and the internal polarization process of ferroelectric materials.

The core solver of this software is DEVSIM. DEVSIM is an open-source semiconductor simulation software; this software is a secondary development built on top of DEVSIM, and provides dedicated solutions for specific types of physical devices.<sup>1</sup>

Based on the finite volume method, DEVSIM constructs the iterative formulations of Gauss's law for the electric field and the carrier continuity equations in semiconductors, and solves them with the Newton iteration method. Its iterative formulations are built mainly in the form of symbolic functions, so the equations of various physical models in electronic devices can be conveniently written into corresponding symbolic functions, which greatly facilitates the implementation and solution of complex physical models. Understanding the basic concepts and usage of DEVSIM is the prerequisite for using this software. For the detailed mathematical principles and usage instructions of DEVSIM, please refer to the website <https://devsim.net/>

---

<sup>1</sup> DEVSIM LLC: DEVSIM TCAD semiconductor device simulator. Available: <https://devsim.org>

## 2. Software Modules

This software currently contains the following functional modules (each module is provided in the form of a Python extension library):

- 1) Create, delete, and check DEVSIM models  
QModel\_create.py
- 2) Create silicon semiconductor physical relationship models  
QSimple\_physics.py
- 3) Create defect state physical relationship models  
QExtraCharge.py
- 4) Create ferroelectric material relationship models  
QSFerro.py
- 5) Create Schottky contact relationship models  
QSSchottkyContact.py
- 6) Execute device voltage sweeps  
QPlotSweep.py
- 7) Create one-dimensional and two-dimensional diode simulation devices  
QDiode\_common.py

### 3. Installation Steps

The operating platform of this software is exactly the same as that of DEVSIM. Python is used as the execution platform, so it can be used on Linux, MacOS, and Windows operating systems. It is recommended to use Conda or MiniConda on Linux as the Python configuration environment.

In addition to DEVSIM, the use of this software also requires the following open-source software for specific pre-processing and post-processing tasks:

1. Multi-dimensional mesh generation software: Gmsh [<https://gmsh.info/>]
2. Data visualization and image generation software: VisIt [<https://www.visitusers.org/>]

The following describes the detailed installation steps, using the Ubuntu Linux operating system and MiniConda as the background.

1. Install MiniConda  
Follow the instructions on the MiniConda website [<https://docs.conda.io/projects/miniconda/en/latest/#>], download the installer for your hardware platform and install it, or install it directly from the command line.
2. Install DEVSIM  
Follow the instructions on the DEVSIM website [<https://devsim.org/introduction.html#download>], download the installer for your hardware platform and install it.
3. Install Gmsh  
Install Gmsh by executing the following command in the Linux system command line.
4. Install VisIt  
Follow the instructions on the VisIt website [<https://www.visitusers.org/>], download the installer for your hardware platform and install it.
5. Required Python Extension Libraries  
The Python extension libraries required by this software mainly include: mkl, numpy, matplotlib, pandas, scipy, sympy, etc. They can be installed by executing the following command in the Linux system command line:
6. Using the Modules of This Software  
It is recommended to place all extension library files of this software in the same path, and import them with the following Python statements:

```
from devsim import *  
import sys, devsim
```

```
sys.path.append('[storage path]')  
from [extension library name] import *
```

## 4. Module and Function Reference

### 1) QSmodel\_create.py

This module is used to create, delete, and check various models in DEVSIM.

CreateSolution(device, region, name)

Create a mesh node variable, which can be used to store node data or to serve as a solution variable of the finite volume method.

Parameters:

device: (string) device name

region: (string) region name

name: (string) name of the variable to be created

CreateNodeModel(device, region, model, expression):

Create a mesh node model (formula)

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the node model to be created

expression: (string) formula of the node model to be created, in the form of a symbolic function

CreateNodeModelDerivative(device, region, model, expression, \*vars):

Create the derivatives of a mesh node model with respect to multiple solution variables

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the node model quantity

expression: (string) formula of the node model to be created, in the form of a symbolic function

\*vars: (string) solution variable name(s)



CreateNodeModelAndDerivative(device, region, model, expression, \*vars):

Create a mesh node model and, at the same time, create its derivatives with respect to the solution variables

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the node model to be created

expression: (string) formula of the node model to be created, in the form of a symbolic function

\*vars: (string) solution variable name(s)

CreateContactNodeModel(device, contact, model, expression):

Create a mesh node model on a contact

Parameters:

device: (string) device name

contact: (string) contact name

model: (string) name of the variable to be created

expression: (string) formula of the node model to be created, in the form of a symbolic function

CreateContactNodeModelDerivative(device, contact, model, expression, \*vars):

Create the derivatives of a mesh node model on a contact with respect to multiple solution variables

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the variable to be created

expression: (string) formula of the node model to be created, in the form of a symbolic function

\*vars: (string) solution variable name(s)

CreateEdgeModel(device, region, model, expression):

Create a mesh edge model

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the edge model to be created

expression: (string) formula of the edge model to be created, in the form of a symbolic function

CreateEdgeModelDerivatives(device, region, model, expression, variable):

Create the derivative of a mesh edge model with respect to a solution variable

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the edge model to be created

expression: (string) formula of the edge model to be created, in the form of a symbolic function

variable: (string) solution variable name

CreateEdgeModelAndDerivatives(device, region, model, expression, \*vars):

Create a mesh edge model and, at the same time, create its derivatives with respect to multiple solution variables

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the edge model to be created

expression: (string) formula of the edge model to be created, in the form of a symbolic function

\*vars: (string) name(s) of multiple solution variables

CreateEdgeFromSqrtNode(device, region, model, \*vars):

Create an edge model from the square root of the product of the node variables at the two ends of a mesh edge, and at the same time create the derivatives of the edge model with respect to multiple solution variables. The name of the created edge model is: `"Sqrt#x201D; + model`

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the referenced node variable

\*vars: (string) solution variable name(s)

`CreateContactEdgeModel(device, contact, model, expression):`

Create a mesh edge model on a contact

Parameters:

device: (string) device name

contact: (string) contact name

model: (string) name of the edge model to be created

expression: (string) formula of the edge model to be created, in the form of a symbolic function

`CreateContactEdgeModelDerivative(device, contact, model, expression, variable):`

Create the derivative of a mesh edge model on a contact with respect to a solution variable

Parameters:

device: (string) device name

contact: (string) contact name

model: (string) name of the edge model to be created

expression: (string) formula of the edge model to be created, in the form of a symbolic function

variable: (string) solution variable name

`CreateInterfaceModel(device, interface, model, expression):`

Create an interface model formula

Parameters:

device: (string) device name

interface: (string) interface name

model: (string) name of the interface model to be created

expression: (string) formula of the edge model to be created, in the form of a symbolic function

CreateContinuousInterfaceModel(device, interface, variable):

Create a continuous interface model formula so that the same node variable varies continuously across the interface. The name of the created interface model is: "continuous&#x201D; + variable

Parameters:

device: (string) device name

interface: (string) interface name

variable: (string) name of the node variable used

InEdgeModelList(device, region, model):

Check whether a specific edge model exists in the specified region; returns a Boolean value

Parameters:

device: (string) device name

region: (string) region name

model: (string) the edge model to be checked

InNodeModelList(device, region, model):

Check whether a specific node model exists in the specified region; returns a Boolean value

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the node model to be checked

InElementModelList(device, region, model):

Check whether a specific element edge model exists in the specified region; returns a Boolean value

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the element edge model to be checked

InSolutionList(device, region, solution):

Check whether a specific node variable exists in the specified region; returns a Boolean value

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the node variable to be checked

InContactList(device, contact):

Check whether a specific contact exists in the specified device; returns a Boolean value

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the contact to be checked

InParameterList(device, Parameter, region=None)

Check whether a specific parameter exists in a specific region of the specified device; returns a Boolean value

Parameters:

device: (string) device name

Parameter: (string) region name

region: (string) region name; if no region is specified, the search is performed among the common parameters

EnsureEdgeFromNodeModelExists(device, region, nodemodel):

Check whether a specific node model exists in a specific region of the specified device; if it does not exist, raise an error. Then use this node model to create two edge models, whose names are respectively: nodemodel + `&#x201C;@n0&#x201D;` and nodemodel + `&#x201C;@n1&#x201D;`

Parameters:

device: (string) device name

Parameter: (string) region name

nodemodel: (string) node model name

CreateDimensionLable(device):

Create the device parameter `&quot;DimList&quot;` for storing coordinate labels. The one-dimensional structure is (`&quot;x&quot;;`), the two-dimensional structure is (`&quot;x&quot;; &quot;y&quot;;`), and the three-dimensional structure is (`&quot;x&quot;; &quot;y&quot;; &quot;z&quot;;`)

Parameters:

device: (string) device name

CreateElementModel2d(device, region, model, expression):

Create a mesh element edge model

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the edge model to be created

expression: (string) formula of the edge model to be created, in the form of a symbolic function

CreateElementModelDerivative2d(device, region, model\_name, expression, \*args):

Create the derivatives of a mesh element edge model with respect to multiple solution variables

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the edge model to be created

expression: (string) formula of the edge model to be created, in the form of a symbolic function

\*args: (string) name(s) of multiple solution variables

CreateElementEdgeFromSqrtNode(device, region, model, \*vars):

Create an element edge model from the square root of the product of the node variables at the two ends of a mesh edge, and at the same time create the derivatives of the element edge model with respect to multiple solution variables. The name of the created edge model is: "Sqrt"; + model

Parameters:

device: (string) device name

region: (string) region name

model: (string) name of the referenced node variable

\*args: (string) name(s) of multiple solution variables

DeleteEdgeModelAndDerivatives(device, region, name):

Delete a specific edge model in the specified region and its derivatives with respect to all node variables

Parameters:

device: (string) device name

region: (string) region name

name: (string) name of the specific edge model

DeleteNodeModelAndDerivatives(device, region, name):

Delete a specific node model in the specified region and its derivatives with respect to all node variables

Parameters:

device: (string) device name

region: (string) region name

name: (string) name of the specific edge model

DeleteNodeModelAndDerivates(device, region, name)

Delete a node model and its derivatives

Parameters:

device: (string) device name

region: (string) region name

name: (string) node model name

## 2) QSimple\_physics.py

Based on the physical relationships of silicon semiconductors under the Boltzmann approximation, create the basic physical models for device simulation

InDisplayHost():

Check whether the host is one for which the running results need to be displayed in real time; returns a Boolean value

InNoneCompressHost():

Check whether the current host is one for which the exported files need to be compressed; returns a Boolean value

SetExtendedPrecision(Threads=4,TaskSize=2048):

Set the number of threads used by DEVSIM at runtime and the number of elements in a single computation

SetVerbosity(verbose=True):

Set whether to output in verbose mode in real time

SetCentiMeterBasicParameters():

Set the basic physical parameters using the centimeter as the basic unit of length, including the elementary charge, the Boltzmann constant, and the vacuum permittivity

SetMicroMeterBasicParameters():



Set the basic physical parameters using the micrometer as the basic unit of length, including the elementary charge, the Boltzmann constant, and the vacuum permittivity

SetOxideParameters(device, region, rPermittivity=1.0):

Set the relative permittivity of the dielectric material

Parameters:

device: (string) device name

region: (string) region name

rPermittivity: (numeric) relative permittivity

SetDopingParameters(device, region, bulk\_doping, drain\_doping, source\_doping):

Set the doping concentrations in the bulk of the transistor active region and near the drain and source

Parameters:

device: (string) device name

region: (string) region name

bulk\_doping: (numeric) bulk doping concentration, default is 0

drain\_doping: (numeric) drain doping concentration, default is 0

source\_doping: (numeric) source doping concentration, default is 0

SetMicroMeterSiliconParameters(device, region, T, rPermittivity, n\_i, nMobility, pMobility, nLifeTime, pLifeTime):

Set the basic physical parameters of a silicon material region using the micrometer as the basic unit of length

Parameters:

device: (string) device name

region: (string) region name

T: (numeric) device temperature, default is 300 K

rPermittivity: (numeric) relative permittivity, default is 11

n\_i: (numeric) intrinsic carrier concentration, default is 1.0e-2

nMobility: (numeric) electron mobility, default is 1e8

pMobility: (numeric) hole mobility, default is 1e8

nLifeTime: (numeric) average electron lifetime, default is 1e-5 s

pLifeTime: (numeric) average hole lifetime, default is 1e-5 s

SetCentiMeterSiliconParameters(device, region, T, rPermittivity, n\_i, nMobility, pMobility, nLifeTime, pLifeTime):

Set the basic physical parameters of a silicon material region using the centimeter as the basic unit of length

Parameters:

device: (string) device name

region: (string) region name

T: (numeric) device temperature, default is 300 K

rPermittivity: (numeric) relative permittivity, default is 11

n\_i: (numeric) intrinsic carrier concentration, default is 1.0e10

nMobility: (numeric) electron mobility, default is 1

pMobility: (numeric) hole mobility, default is 1

nLifeTime: (numeric) average electron lifetime, default is 1e-5 s

pLifeTime: (numeric) average hole lifetime, default is 1e-5 s

SetGaussianDOSCarrierParameters(device, region, TypeName, BulkDensity, Width, Center, Rate0, LocalScale, HDistance):

Set the basic parameters of Gaussian-distributed energy levels

Parameters:

device: (string) device name

region: (string) region name

TypeName: (string) energy level type, options are "GaussianHomo" or "GaussianLumo";

BulkDensity: (numeric) total density of states

Width: (numeric) width of the energy level distribution

Center: (numeric) center position of the energy level

Rate0: (numeric) attempt-to-transition frequency

LocalScale: (numeric) localization scale

HDistance: (numeric) average hopping distance of carriers within the energy level

CreateGaussianCarriersExchange(device, region, BasicRate, Distance):

Set the carrier transition model between the conduction band and the valence band with Gaussian distribution

Parameters:

device: (string) device name

region: (string) region name

BasicRate: (numeric) attempt-to-transition frequency

Distance: (numeric) average transition distance

CreatePotentialAndFlux(device, region)

Set the potential and electric field models

Parameters:

device: (string) device name

region: (string) region name

CreateSiliconNetDoping(device, region, bulk\_doping)

Set the net doping concentration of a silicon material region

Parameters:

device: (string) device name

region: (string) region name

bulk\_doping: (numeric) net doping concentration

CreateSiliconPotentialOnly(device, region):

Calculate the potential and carrier concentrations of the device in the equilibrium state, to be used as initial values

Parameters:

device: (string) device name

region: (string) region name

DeleteIntrinsicNodeModelDerivatives(device, region):

Delete the redundant variables and models used during the initial value calculation

Parameters:

device: (string) device name

region: (string) region name

CreateSiliconPotentialOnlyContact(device, contact, is\_circuit=False, offset=False)

Set the potential at the contact in the equilibrium state, to be used as the initial value

Parameters:

device: (string) device name

contact: (string) contact name

is\_circuit: (Boolean) whether the contact is connected to a circuit

offset: (Boolean) whether the potential is offset

CreateSRH(device, region):

Set the Shockley-Read-Hall (SRH) recombination model in the semiconductor region

CreateECE(device, region, mu\_n, Bandtype="Silicon")

Set the electron current model and its formula in the semiconductor region

Parameters:

device: (string) device name

contact: (string) contact name

mu\_n: (numeric) mobility model

Bandtype: (string) band model, options are "Gaussian" or "Silicon";

CreateHCE(device, region, mu\_p, Bandtype="Silicon")

Set the hole current model and its formula in the semiconductor region

Parameters:

device: (string) device name

contact: (string) contact name

mu\_n: (numeric) mobility model

Bandtype: (string) band model, options are "Gaussian" or "Silicon";

CreatePE(device, region):

Set the Gauss's law formula for the electric field in the semiconductor region

Parameters:

device: (string) device name

contact: (string) contact name

mu\_n: (numeric) mobility model

Bandtype: (string) band model, options are "Gaussian" or "Silicon";

CreateElectronAndHoleSolutions(device, region, PreValue=True, Bandtype="Silicon")

Create the electron and hole solution variables in the semiconductor region, and choose according to PreValue whether to obtain the initial values from the equilibrium state

Parameters:

device: (string) device name

contact: (string) contact name

PreValue: (numeric) mobility model

Bandtype: (string) band model, options are "Silicon";

SetInitialSolutionFromDopping(device, region, Bandtype="Silicon")

Set the initial values of the electron and hole solution variables according to the doping concentration

Parameters:

device: (string) device name

contact: (string) contact name

PreValue: (numeric) mobility model

Bandtype: (string) band model, options are "Silicon";

CreateSiliconDriftDiffusion(device, region, mu\_n, mu\_p="mu\_p\_eff", SRHMolde=True)

Create the electron and hole continuity equations in the semiconductor region

Parameters:

device: (string) device name

contact: (string) contact name

mu\_n: (numeric) electron mobility model

mu\_p: (numeric) hole mobility model

SRHModel: (Boolean) whether to use the Shockley-Read-Hall (SRH) recombination model

CreateSiliconDriftDiffusionAtContact(device, contact, is\_circuit=False)

Create the electron and hole continuity equations at the contact of the semiconductor region; the equilibrium concentration over the entire contact can vary with the doping

Parameters:

device: (string) device name

contact: (string) contact name

is\_circuit: (Boolean) whether the contact is connected to a circuit

CreateElectronAndHoleAtContact(device, contact, is\_circuit=False)

Create the electron and hole continuity equations at the contact of the semiconductor region; the equilibrium concentration over the entire contact is uniform

Parameters:

device: (string) device name

contact: (string) contact name

is\_circuit: (Boolean) whether the contact is connected to a circuit

CreateOxidePotentialOnly(device, region, update\_type="auct"):

Create the Gauss's law equation for the electric field in the dielectric region, where the electric field vector is an edge quantity

Parameters:

device: (string) device name

region: (string) dielectric region name

update\_type: (Boolean) variable update mode

CreateElementOxidePotentialOnly(device, region, update\_type="ault"):

Create the Gauss's law equation for the electric field in the dielectric region, where the electric field vector is an element edge quantity

Parameters:

device: (string) device name

region: (string) dielectric region name

update\_type: (Boolean) variable update mode

CreateOxidePotentialContact(device, contact, element\_contact=False, attached\_to=None, offset\_contact=None, is\_circuit=False)

Create the boundary condition of the electric field in the dielectric region at the contact

Parameters:

device: (string) device name

contact: (string) contact name

element\_contact: (Boolean) whether the electric field vector uses element edges; the default is `No`, i.e., edges are used

attached\_to: (string) whether it is linked to another contact; the default is `None`;

offset\_contact: (string) whether the potential is offset together with that of another contact

is\_circuit: (Boolean) whether the contact is connected to a circuit

CreateSiliconOxideInterface(device, interface, InterfaceType="continuous"):

Create the interface model between the semiconductor region and the dielectric region

Parameters:

device: (string) device name

interface: (string) interface name

InterfaceType: (Boolean) interface matching mode of Gauss's law for the electric field; the default is continuous

CreateSiliconSiliconInterface(device, interface):

Create the electron and hole continuity equations as well as Gauss's law for the electric field in continuous mode

CheckNodeValues(device, region, NodeChecklist=None, Length=5)

Print the values of node variables and models

Parameters:

device: (string) device name

region: (string) dielectric region name

NodeChecklist: (list) names of the variables and models

Length: (numeric) number of values printed for each variable and model

CheckElementValues(device, region, ElementChecklist=None, Length=5):

Print the values of element edges

Parameters:

device: (string) device name

region: (string) dielectric region name

ElementChecklist: (list) names of the element edges

Length: (numeric) number of values printed for each variable and model

CheckEdgeValues(device, region, EdgeChecklist=None, Length=5):

Print the values of edges

Parameters:

device: (string) device name

region: (string) dielectric region name

EdgeChecklist: (list) names of the edges

Length: (numeric) number of values printed for each variable and model

CreateAbsEfield(device, region):

Create the absolute value of the electric field; the element edge

&#x201C;ElectricField\_mag&#x201D; is created, which is suitable for plotting

Parameters:

device: (string) device name



region: (string) dielectric region name

CreateAbsCurrentAnield(device, region, magnitude=False):

Create the carrier flow velocity and the absolute value of the electric field

Parameters:

device: (string) device name

region: (string) dielectric region name

ElectricFieldMagnitude(device, region):

Create the absolute value of the electric field; create the element edge  $\nabla \cdot \text{ElectricField\_mag}$ ; and its derivative with respect to the potential, which is suitable for mobility models

Parameters:

device: (string) device name

region: (string) dielectric region name

MicorSaturationMobilityParameters(device, region, V\_n\_Sat=1.0e11, V\_p\_Sat=1.0e11, LorentzEField=1.0e-2):

Create or modify the parameters used by the saturation mobility model in micrometer units

Parameters:

device: (string) device name

region: (string) dielectric region name

V\_n\_Sat: (numeric) electron saturation velocity, default is 1.0e11

V\_p\_Sat: (numeric) hole saturation velocity, default is 1.0e11

LorentzEField: (numeric) Lorentz saturation field. Default = 1.0e-2

CentiSaturationMobilityParameters(device, region, V\_n\_Sat=1.0e7, V\_p\_Sat=1.0e7, LorentzEField=1.0e4):

Create the saturation mobility model in centimeter units

Parameters:

device: (string) device name

region: (string) dielectric region name

V\_n\_Sat: (numeric) electron saturation velocity, default is 1.0e7

V\_p\_Sat: (numeric) hole saturation velocity, default is 1.0e7

LorentzEField: (numeric) Lorentz saturation field. Default = 1.0e4

CreateNormalElectricFieldFromCurrentFlow(device, region, low\_curr):

Create the component of the electric field along the specified carrier current

Parameters:

device: (string) device name

region: (string) dielectric region name

low\_curr: (string) the specified carrier current

ProjectSaturationMobility(device, region, ChargeType):

Create a saturation mobility model

Parameters:

device: (string) device name

region: (string) dielectric region name

ChargeType: (string) carrier type, options are "Electrons" or  
"Holes";

QSSaturationMobility(device, region, ChargeType, EfieldModel="ElectricField\_x"):

Create a unidirectional saturation mobility model

Parameters:

device: (string) device name

region: (string) dielectric region name

ChargeType: (string) carrier type, options are "Electrons" or  
"Holes";

EfieldModel: (string) selected electric field direction, options are  
"ElectricField\_x" or "ElectricField\_y";

LorentzSaturationMobility(device, region, mu\_sat, LorentzEField, ChargeType):

Create a Lorentz saturation mobility model

Parameters:

device: (string) device name

region: (string) dielectric region name

mu\_sat: (numeric) magnitude of the saturation mobility

LorentzEField: (string) selected electric field direction, options are  
&quot;ElectricField\_x&quot; or &quot;ElectricField\_y&quot;;

ChargeType: (string) carrier type, options are &quot;Electrons&quot; or  
&quot;Holes&quot;;

PooleFrenkelMobilityParameters(device, region, ChargeType, ActivePotential=0,  
Beta=0.1, Gama=0)

Create or modify the parameters used by the Poole-Frenkel mobility model

Parameters:

device: (string) device name

region: (string) dielectric region name

ChargeType: (string) carrier type, options are &quot;Electrons&quot; or  
&quot;Holes&quot;;

ActivePotential: (numeric) activation barrier

Beta: (numeric)

Gama: (numeric)

PooleFrenkelMobility(device, region, ChargeType):

Create a Poole-Frenkel mobility model

Parameters:

device: (string) device name

region: (string) dielectric region name

ChargeType: (string) carrier type, options are &quot;Electrons&quot; or  
&quot;Holes&quot;;

CreateElementGaussianQuasiFermi(device, region, TypeName, CarrierName):

Create the element edge model of the Fermi level of carriers with a Gaussian Fermi level distribution from the carrier concentration; the electrostatic potential is not included.

Parameters:

device: (string) device name

region: (string) dielectric region name

TypeName: (string) energy level structure type, options are

&quot;GaussianDLkT&quot;, &quot;GaussianHomo&quot;,  
&quot;GaussianALkT&quot;, &quot;GaussianLumo&quot;

CarrierName: (string) carrier type, options are &quot;Electrons&quot; or  
&quot;Holes&quot;

CreateEdgeGaussianQuasiFermi(device, region, TypeName, CarrierName):

Create the edge model of the Fermi level of carriers with a Gaussian Fermi level distribution from the carrier concentration; the electrostatic potential is not included.

Parameters:

device: (string) device name

region: (string) dielectric region name

TypeName: (string) energy level structure type, options are

&quot;GaussianDLkT&quot;, &quot;GaussianHomo&quot;,  
&quot;GaussianALkT&quot;, &quot;GaussianLumo&quot;

CarrierName: (string) carrier type, options are &quot;Electrons&quot; or  
&quot;Holes&quot;

CreateNodeGaussianQuasiFermi(device, region, TypeName, SolutionName=None):

Create the node model of the Fermi level of carriers with a Gaussian Fermi level distribution from the carrier concentration; the electrostatic potential is not included.

Parameters:

device: (string) device name

region: (string) dielectric region name

TypeName: (string) energy level structure type, options are

&quot;GaussianDLkT&quot;, &quot;GaussianHomo&quot;,  
&quot;GaussianALkT&quot;, &quot;GaussianLumo&quot;

CarrierName: (string) carrier type, options are &quot;Electrons&quot; or  
&quot;Holes&quot;

MacCDTMobility(device, region, CarrierName, MacCDTPre=2e-9, MacCo=1,  
PooleFrenkel=0.26, TrapName=None, MiniValue=1e2, TailState=1e-8):

Create a MacCDT mobility model

Parameters:

device: (string) device name

region: (string) dielectric region name

CarrierName: (string) carrier type, options are "Electrons" or  
"Holes";

ActivePotential: (numeric) activation barrier

MacCDTPre: (numeric) parameter, default is 2e-9

MacCo: (numeric) parameter, default is 1

PooleFrenkel: (numeric) parameter, default is 0.26

TrapName: (string) trap state type, empty by default

MiniValue: (numeric) parameter, default is 1e2

TailState: (numeric) parameter, default is 1e-8

ShurHackMobility(device, region, Carrier, TrapName, LRatio=0.01,Exponent=1):

Create a Shur-Hack mobility model

Parameters:

device: (string) device name

region: (string) dielectric region name

Carrier: (string) carrier type, options are "Electrons" or  
"Holes";

TrapName: (string) trap state type, empty by default

LRatio: (numeric) parameter, default is 0.01

Exponent: (numeric) parameter, default is 1

ElementElectronCurrent2d(device, region)

Create the element edge model of the electron current

Parameters:

device: (string) device name

region: (string) dielectric region name

ElementElectronContinuityEquation(device, region):

Create the electron continuity equation from the element edge model of the electron current

Parameters:

device: (string) device name

region: (string) dielectric region name

ElementHoleCurrent2d(device, region, OrganicCurrent=False):

Create the element edge model of the hole current

Parameters:

device: (string) device name

region: (string) dielectric region name

ElementHoleContinuityEquation(device, region):

Create the element edge model of the hole current

Parameters:

device: (string) device name

region: (string) dielectric region name

CreateElementSiliconDriftDiffusion(device, region):

Create the carrier drift-diffusion model based on element edges

Parameters:

device: (string) device name

region: (string) dielectric region name

ElementContactElectronContinuityEquation(device, contact):

Create the electron boundary condition based on element edges at the contact

Parameters:

device: (string) device name

region: (string) dielectric region name

ElementContactHoleContinuityEquation(device, contact):

Create the hole boundary condition based on element edges at the contact

Parameters:

device: (string) device name

region: (string) dielectric region name

CreateElementCurrentContact(device, contact):

Create the current boundary condition based on element edges at the contact

Parameters:

device: (string) device name

region: (string) dielectric region name

CreateCapacitorCircuit(contact, attached\_to=None, value=0.0, aereal=0.0, resistance=1e0)

Create the capacitor circuit structure relationship

Parameters:

contact: (string) capacitor name

attached\_to: (string) whether it is linked to another contact; the default is  
&#x201C;None&#x201D;

value: (numeric) capacitance, default is 0.0

aereal: (numeric) real part of the AC voltage, default is 0.0

resistance: (numeric) magnitude of the series resistance, default is 1

CreateContactCircuit(contact, attached\_to=None, value=0.0, aereal=0.0, resistance=1e0)

Create the capacitor circuit structure relationship

Parameters:

contact: (string) capacitor name

attached\_to: (string) whether it is linked to another contact; the default is  
&#x201C;None&#x201D;

value: (numeric) capacitance, default is 0.0

aereal: (numeric) real part of the AC voltage, default is 0.0

resistance: (numeric) magnitude of the series resistance, default is 1

CreateTransientCircuit(contact, resistance=1e0):

Add a series resistance to the contact for transient analysis

Parameters:

contact: (string) capacitor name

resistance: (numeric) magnitude of the series resistance, default is 1

PrintCapacitor(device, contact, frequency=[1.0,]):

Print the capacitance of a contact in the device at a specific frequency

Parameters:

device: (string) device name

contact: (string) capacitor name

frequency: (list) frequencies to be tested

GetSaveFileName(device,FileName):

Obtain the path and file name of the saved file from the object &quot;Description&quot; in the device

Parameters:

device: (string) device name

FileName: (string) preset file name

QSSaveDevice(device, file, ftype="devsim"):

Save the device simulation results; GetSaveFileName is called to generate the path

Parameters:

device: (string) device name

file: (string) file name

ftype: (string) save type, options are 'devsim', 'devsim\_data', 'floops', 'tecplot', 'vtk'; the default is &quot;devsim&quot;;

ExportParameters(device, file="DeviceParameters")

Output all device parameters as a Python file; GetSaveFileName is called to generate the path



Parameters:

device: (string) device name

file: (string) file name, default is "DeviceParameters";

GetContactBiasName(contact):

Get the name of the contact potential

Parameters:

contact: (string) contact name

GetContactNodeModelName(contact):

Get the name of the contact node model

Parameters:

contact: (string) contact name

PrintCurrents(device, contact, ContactList=[]):

Get the current of a device contact

Parameters:

device: (string) device name

contact: (string) capacitor name

ContactList: (list) list of contact names

SetContactDopingOffset(device, contact):

Based on the Boltzmann model of silicon semiconductors, set the initial potential condition of the ohmic contact at the contact from the net doping concentration

Parameters:

device: (string) device name

contact: (string) capacitor name

GetContactDopingOffset(device, contact):

Get the potential offset of the ohmic contact at the contact caused by the net doping concentration

Parameters:

device: (string) device name

contact: (string) capacitor name

SetContactPotentialOffset(device, contact, PotentialOffset):

Set the initial potential condition of the ohmic contact at the contact

Parameters:

device: (string) device name

contact: (string) capacitor name

PotentialOffset: (numeric) potential offset

GetContactPotentialOffset(device, contact):

Get the potential offset of the contact

Parameters:

device: (string) device name

contact: (string) capacitor name

SpecificValueInitiateRamp(device, Region, Parameter, abs\_error=1e30, rel\_error=1e-8, iterations=30, ratio=2):

Calculate the equilibrium state after the parameter is introduced; if equilibrium cannot be reached, divide the parameter by a certain factor and calculate again

Parameters:

device: (string) device name

region: (string) region name

Parameter: (string) region name

abs\_error: (numeric) absolute value of the iteration error, default is 1e30

rel\_error: (numeric) relative value of the iteration error, default is 1e-8

iterations: (numeric) iteration count limit, default is 30

ratio: (numeric) the factor by which the parameter is divided, default is 2

SpecificValueApproach(device, Region, Parameter, SetToValue, abs\_error=1e30, rel\_error=1e-8, iterations=30, step\_lag=0.1, min\_setp=0.001, method="Line"):

Change a specific parameter with a certain step size and calculate the equilibrium state; if equilibrium cannot be reached, divide the parameter by 2 and calculate again, until the step size becomes smaller than a specific value and the attempt is abandoned

Parameters:

device: (string) device name

region: (string) region name

Parameter: (string) region name

abs\_error: (numeric) absolute value of the iteration error, default is 1e30

rel\_error: (numeric) relative value of the iteration error, default is 1e-8

iterations: (numeric) iteration count limit, default is 30

ratio: (numeric) the factor by which the parameter is divided, default is 2

step\_lag: (numeric) step size, default is 0.1

min\_setp: (numeric) minimum step size, default is 0.001

method: (numeric) approach mode, default is &quot;Line&quot;;

InitialSolve(device, orders=5, type="dc", abs\_error=1e30, rel\_error=1e-8, iterations=30, tdelta=0.0)

Calculate the equilibrium state; if equilibrium cannot be reached, increase the relative error tolerance and calculate again, until the relative error has increased by a power of 10 and the attempt is abandoned

Parameters:

device: (string) device name

orders: (numeric) the limited exponent, default is 5

type: (string) calculation mode, default is DC &quot;dc&quot;;

abs\_error: (numeric) absolute value of the iteration error, default is 1e30

rel\_error: (numeric) relative value of the iteration error, default is 1e-8

iterations: (numeric) iteration count limit, default is 30

tdelta: (numeric) time step used for transient simulation, default is 0.0

NodeModelVolumeIntegrator(device, NodeDic)

Calculate the integral value of a specific mesh node model, or apply other calculation methods

Parameters:

device: (string) device name

NodeDic: (dictionary) contains three values: the region name "region", the node name "NodeName", and the calculation method "SampleType", whose options are volume integral "VolSum", maximum value "Max", and minimum value "Min";

### 3) QSEExtraCharge.py

Create defect state physical relationship models

CreateGaussianDOS(device, region, Type, Width, Center, BulkDensity, Interface=None, Thickness=0.05)

Set the basic parameters of Gaussian-distributed energy levels

Parameters:

device: (string) device name

region: (string) region name

Type: (string) energy level type, options are "GaussianHomo" or "GaussianLumo";

Width: (numeric) width of the energy level distribution

Center: (numeric) center position of the energy level

BulkDensity: (numeric) total density of states

Interface: (numeric) interface name, empty by default; if it is given, an interface state is created

Thickness: (numeric) thickness of the interface state, default is 0.05

FixExtraNodeCharge(device, region, Charge\_Name):

Convert a node model charge into a node variable

Parameters:

device: (string) device name

region: (string) region name

Charge\_Name: (string) name of the node model charge

InitDetrappingGaussianCharge(device, region, Charge\_Name, Type):

Set the charge concentration of Gaussian-distributed trap states

Parameters:

device: (string) device name

region: (string) region name

Charge\_Name: (string) name of the node model charge

Type: (string) the selected trap state type

SetGaussianDOSTrapParameters(device, region, TrapName, TotalDensity, Width, Center, Rate0, LocalScale, HDistance, Interface=None, Thickness=0.001):

Parameters:

device: (string) device name

region: (string) region name

TrapName: (string) trap state name

TotalDensity: (numeric) total density of states

Width: (numeric) width of the energy level distribution

Center: (numeric) center position of the energy level

Rate0: (numeric) attempt-to-transition frequency

LocalScale: (numeric) localization scale

HDistance: (numeric) average hopping distance of carriers within the energy level

Interface: (numeric) interface name, empty by default; if it is given, an interface state is created

Thickness: (numeric) thickness of the interface state, default is 0.05

CreateDonorLikeGaussianTrapEquation(device, region, PooleFrenkel=True, LumoRate=False):

Set the formula of Gaussian-distributed trap states

Parameters:

device: (string) device name

region: (string) region name

PooleFrenkel: (Boolean) whether to use the Poole-Frenkel model, default is True

LumoRate: (Boolean) whether to calculate the transition relationship with the LUMO orbital, default is False

CreateGaussianTrapNode(device, region, TrapName, LumoRate=False, HomoRate=False, PooleFrenkel=True):

Set the node model of Gaussian-distributed trap states

Parameters:

device: (string) device name

region: (string) region name

PooleFrenkel: (Boolean) whether to use the Poole-Frenkel model, default is True

LumoRate: (Boolean) whether to calculate the transition relationship with the LUMO orbital, default is False

HomoRate: (Boolean) whether to calculate the transition relationship with the LUMO orbital, default is False

CreateDonorLikeGaussianTrapContactEquation(device, region, contact)

Set the contact formula of Gaussian-distributed trap states

Parameters:

device: (string) device name

region: (string) region name

contact: (string) contact name

CreateGaussianDOSDetrapping(device, region, ChargeModel, ChargeName, DetrappingDistance, Interface):

Set the recombination model of Gaussian-distributed trap states

Parameters:

device: (string) device name

region: (string) region name

ChargeModel: (string) charge type

ChargeName: (string) charge name

DetrappingDistance: (string)

Interface: (string) interface name

ReplaceExtraPotentialNodeCharge(device, region, \*Variables):

Replace the non-carrier part of the charge in Gauss's law for the electric field

Parameters:

device: (string) device name

region: (string) region name

Variables: (string) names of multiple charges

InterfaceCharge\_NodeModel(device, region, Interface, Charge\_Type, Thickness):

Set the interface charge node model

Parameters:

device: (string) device name

region: (string) region name

Interface: (string) interface name

Charge\_Type: (string) charge type

Thickness: (numeric) thickness of the interface state

Interface\_NodeParameter(device, region, Interface, ParameterName, Thickness):

Set the interface node model parameters

Parameters:

device: (string) device name

region: (string) region name

Interface: (string) interface name

ParameterName: (string) parameter name

Thickness: (numeric) thickness of the interface state

#### 4) QSFerro.py

Create ferroelectric material relationship models

CentiMeterFerroParameters(device, region,

SaturationPolarization =  $7.749 \times 10^{-6}$ , #C/cm<sup>2</sup>

RemanentPolarization = 7.130e-6, #C/cm^2

CoerciveField = 5.0e5, #V/cm

eps\_Ferro = 12,

FunctionModel = "tanh"):

Set the ferroelectric dielectric parameters of the Miller model, using the centimeter as the basic unit of length

Parameters:

device: (string) device name

region: (string) region name

SaturationPolarization: (numeric) saturation polarization, default is 7.749e-6, #C/cm^2

RemanentPolarization: (numeric) remanent polarization, default is 7.130e-6, #C/cm^2

CoerciveField: (numeric) coercive field, default is 5.0e5, #V/cm

eps\_Ferro: (numeric) permittivity, default is 12

FunctionModel: (string) mathematical function, default is "tanh";

MicroMeterFerroParameters(device, region,

SaturationPolarization = 7.749e-14, #C/um^2

RemanentPolarization = 7.130e-14, #C/um^2

CoerciveField = 5e1, #V/um

eps\_Ferro = 12,

FunctionModel = "tanh"):

Set the ferroelectric dielectric parameters of the Miller model, using the micrometer as the basic unit of length

Parameters:

device: (string) device name

region: (string) region name



SaturationPolarization: (numeric) saturation polarization, default is 7.749e-14, #C/um<sup>2</sup>

RemanentPolarization: (numeric) remanent polarization, default is 7.130e-14, #C/um<sup>2</sup>

CoerciveField: (numeric) coercive field, default is 5e1, #V/um

eps\_Ferro: (numeric) permittivity, default is 12

FunctionModel: (string) mathematical function, default is &quot;tanh&quot;;

WriteFerroParameters(device, region, SaturationPolarization, RemanentPolarization, CoerciveField, eps\_Ferro, FunctionModel)

Write the ferroelectric dielectric parameters of the Miller model

Parameters:

device: (string) device name

region: (string) region name

SaturationPolarization: (numeric) saturation polarization

RemanentPolarization: (numeric) remanent polarization

CoerciveField: (numeric) coercive field

eps\_Ferro: (numeric) permittivity

FunctionModel: (string) mathematical function

CreateFerroRegion(device, region, update\_type="aolt", FunctionModel="tanh"):

Set the formula of the ferroelectric region

Parameters:

device: (string) device name

region: (string) region name

update\_type: (string) variable update mode, default is &quot;aolt&quot;;

FunctionModel: (string) mathematical function, default is &quot;tanh&quot;;

CreateFerroContactEquation(device, contact, attached\_to=None, is\_circuit=False):

Set the contact condition of the ferroelectric region

Parameters:

device: (string) device name

region: (string) region name

attached\_to: (string) whether the potential varies with another contact; the default is &quot;a&quot;;

is\_circuit: (Boolean) whether the contact is connected to a circuit; the default is False

FerroRegionIterate(device, region, ElementChecklist=None):

Set the iteration model of the ferroelectric region

Parameters:

device: (string) device name

region: (string) region name

ElementChecklist: (list) list of element edges to be output

GinzburgLandauFerroParameters(device, region,

GLFerro\_rho = 0, #

GLFerro\_alpha2 = 0\*1e4 , # cm/F to um/F

GLFerro\_alpha4 = 0\*1e20, # cm5/FC2 to um5/FC2

):

Set the parameters of the Landau-Ginzburg ferroelectric model

Parameters:

device: (string) device name

region: (string) region name

GLFerro\_rho: (numeric) = 0, #

GLFerro\_alpha2: (numeric) parameter, default is 0\*1e4 um/F

GLFerro\_alpha4: (numeric) parameter, default is 0\*1e20 um5/FC2

CreateGinzburgLandauFerroRegion(device, region, update\_type="a&quot;):

Set the formula of the ferroelectric region

Parameters:

device: (string) device name

region: (string) region name

update\_type: (string) variable update mode, default is &quot;a&quot;;

FunctionModel: (string) mathematical function, default is &quot;tanh&quot;;

PolarizationRamp(device, FerroRegion, abs\_error=1e30, rel\_error=1e-8, iterations=30):

Calculate the initial equilibrium state of the ferroelectric parameters

Parameters:

device: (string) device name

FerroRegion: (string) region name

abs\_error: (numeric) absolute value of the iteration error, default is 1e30

rel\_error: (numeric) relative value of the iteration error, default is 1e-8

iterations: (numeric) iteration count limit, default is 30

CapacitorPlotSweep(device, region,

SweepContact,

ChargeContact,

CurrentContact=None,

End\_bias=1.0,

step\_limit=0.1,

min\_step=0.001,

rel\_error=1e-8,

abs\_error=1e30,

iterations=30,

SaveAs=None,

Checklist=None,

frequency=1e0)

Capacitor voltage sweep

Parameters:

device: (string) device name

region: (string) ferroelectric region name

SweepContact: (string) name of the swept contact

ChargeContact: (string) name of the contact whose charge needs to be output

CurrentContact: (string) name of the contact whose current needs to be calculated, default is None

End\_bias: (numeric) end voltage, default is 1.0

step\_limit: (numeric) initial voltage step size, default is 0.1

min\_step: (numeric) minimum voltage step size, default is 0.001

abs\_error: (numeric) absolute value of the iteration error, default is 1e30

rel\_error: (numeric) relative value of the iteration error, default is 1e-8

iterations: (numeric) iteration count limit, default is 30

SaveAs: (string) None

Checklist: (string) None

frequency: (string) 1e0

#### 5) [QSSchottkyContact.py](#)

Create Schottky contact relationship models

`gfi(zeta, s)`

Gaussian Fermi integral

Parameters:

device: (numeric) variable 1

region: (string) variable 2

`difgfi(zeta,s):`

First-order partial derivative of the Gaussian Fermi integral

Parameters:

zeta: (numeric) variable 1

s: (numeric) variable 2

`gfiinv(g,s):`

Inverse function of the Gaussian Fermi integral

Parameters:

g: (numeric) variable 1

s: (numeric) variable 2

SetSchottkyContactParameters2(device, region, contact, ThermVelocities\_e=1e11, ThermVelocities\_h=1e11)

Set the Schottky contact parameters

Parameters:

device: (string) device name

region: (string) region name

contact: (string) contact name

ThermVelocities\_e: (numeric) electron thermal excitation velocity, default is 1e11

ThermVelocities\_h: (numeric) hole thermal excitation velocity, default is 1e11

SetSchottkyContactParameters(device, region, contact, ThermVelocities\_h=1e11, ThermVelocities\_e=1e11, Bandtype="Silicon"):

Set the Schottky contact parameters

Parameters:

device: (string) device name

region: (string) region name

contact: (string) contact name

ThermVelocities\_e: (numeric) electron thermal excitation velocity, default is 1e11

ThermVelocities\_h: (numeric) hole thermal excitation velocity, default is 1e11

Bandtype: (string) band structure type, default is "Silicon";

CreateSchottkyDriftDiffusionAtContact(device, contact, element\_contact=False):

Set the Schottky contact boundary condition

Parameters:

device: (string) device name

contact: (string) contact name

element\_contact: (Boolean) whether the current vector is an element edge, default is False

CreateSchottkyElementElectronsContactEquation(device, contact):

Set the electron Schottky contact boundary condition with the element edge current vector

Parameters:

device: (string) device name

contact: (string) contact name

CreateSchottkyElementHolesContactEquation(device, contact)

Set the hole Schottky contact boundary condition with the element edge current vector

Parameters:

device: (string) device name

contact: (string) contact name

CreateSchottkyPotentialContact(device, contact, PotentialOffset):

Parameters:

device: (string) device name

region: (string) region name

contact: (string) contact name

SetSchottkEquiCharge(device, region, contact):

Set the equilibrium concentration of the Schottky contact from the current carrier concentration

Parameters:

device: (string) device name

region: (string) region name

contact: (string) contact name

## 6) QSPLOT Sweep.py

Execute the device voltage sweep; the sweep results are displayed in real time and saved as figures using the matplotlib library

BackGroundPlot():

Disable real-time display, save the figures only, and set the display character width of the pandas library

ForGroundPlot():

Disable real-time display and save the figures only

CreatePlotFigures(name, SubFigureNums=1, figsize=(10, 5),

Create a matplotlib figure canvas and its subfigures, and return the objects

Parameters:

name: (string) canvas name

SubFigureNums: (integer) number of subfigures

figsize: (tuple) canvas size

CreateMainFigure(name, figsize=(10, 8)):

Create a matplotlib figure canvas and return the object

Parameters:

name: (string) canvas name

figsize: (tuple) canvas size

CreateSubPlot(Figure, Row, Column, num, axeslabel=["Voltage", "Current"],  
xlog=False, ylog=False, title= None):

Plot in a matplotlib subfigure and return the curve object

Parameters:

Figure: (object) canvas object

Row: (integer) canvas size

Column: (integer) canvas size

num: (integer) canvas size

axeslabel: (list of strings) axis labels, default is ["Voltage",  
"Current"]

xlog: (Boolean) whether the horizontal axis is displayed on a logarithmic scale,  
default is False

ylog: (Boolean) whether the vertical axis is displayed on a logarithmic scale, default  
is False

title: (string) subfigure name, default is None

RefreshSubPlot(SubFigure,Curves,lists,Title=None,ylogScale=False):

Update the subfigure

Parameters:

SubFigure: (object) subfigure object

Curves: (object) curve object

lists: (list) curve array

Title: (string) subfigure name, default is None

ylogScale: (Boolean) whether the vertical axis is displayed on a logarithmic scale, default is False

CreatePlotCurves(Figure, SubFigure, lists=([0.1,0.2],[0.1,0.2]), ylogScale=False,labels=[]):

Plot the curve

Parameters:

Figure: (object) canvas object

SubFigure: (object) subfigure object

lists: (list) curve data

ylogScale: (Boolean) whether the vertical axis is displayed on a logarithmic scale, default is False

labels: (list of strings) curve labels

SavePlotFigures(Figure, FileName, device=None, FileTypes=["pdf"]):

Save the figure

Parameters:

Figure: (object) canvas object

FileName: (string) file name

device: (string) device name, default is None

FileTypes: (list of strings) file types for saving, default is ["pdf"]

RemovePlotFigures(Figure, FileName, device=None, FileTypes=["svg","pdf"],):

Delete the saved figure

Parameters:

Figure: (object) canvas object



FileName: (string) file name, default is None

device: (string) device name

FileTypes: (list of strings) file types for saving, default is ["svg",  
"pdf"]

UpdateExtraNodeCharge(device, Step\_size=0.1, NodeChecklist=None, Length=10)

Update the node variable charge in Gauss's law

Parameters:

device: (string) device name

Step\_size: (numeric) iteration voltage interval

NodeChecklist: (list of node variables) node variables to be displayed before and  
after the update; none by default

Length: (integer) number of node variables to be displayed before and after the  
update, default is 10

VoltagePlotSweep(device,

    SweepModel = ["Current"],

    solvetype = "dc",

    SweepContact = None,

    YlogScale = True,

    CurrentContacts = [],

    ChargeContacts = [],

    End\_bias = 1.0,

    step\_limit = 0.1,

    min\_step = 0.01,

    rel\_error = 1e-8,

    abs\_error = 1e30,

    iterations = 30,

    FerroRegion = None,

    ElementChecklist= None,

    NodeChecklist = None,

```

Lable      = "",
LowBodyBias = False,
CapacitorContacts = [],
frequency  = [1.0,],
VolumeIntegrateList = [],
DeviceMonitorList = [],
SaveAll     = False,
SaveData    = True,
SaveFinal   = True,
SaveZero    = False,
SaveCurrentVariation=0,
SaveRange   = [0,0],
PlotResults = True,

ExtraNodeChargeUpdate= True):

```

Sweep the specified contact of a specific device in linear voltage mode, and output the variations of the current, etc., with voltage

Parameters:

device: (string) device name

SweepModel: (list of strings) device properties output by the sweep, default is ["Current"]; it can also contain "Capacitor" and "NodeCharge";

solvetype: (string) voltage sweep mode, default is "dc";

SweepContact: (string) name of the swept contact, default is None

YlogScale: (Boolean) whether the vertical axis is displayed on a logarithmic scale, default is True

CurrentContacts: (list of strings) contacts whose current is output, empty by default

ChargeContacts: (list of strings) contacts whose charge is output, empty by default

CapacitorContacts: (list of strings) contacts whose capacitance is output, empty by default

End\_bias: (numeric) end voltage, default is 1.0

step\_limit: (numeric) initial voltage step size, default is 0.1

min\_step: (numeric) minimum voltage step size, default is 0.01

rel\_error: (numeric) limit of the relative iteration error, default is 1e-8

abs\_error: (numeric) limit of the absolute iteration error, default is 1e30

iterations: (numeric) iteration count limit, default is 30

FerroRegion: (string) ferroelectric region name, default is None

ElementChecklist: (list) list of element edges to be output at each iteration, default is None

NodeChecklist: (string) list of node models to be output at each iteration, default is None

Lable: (string) additional label of the output file, empty by default

LowBodyBias: (string) if a body contact exists, whether it varies together with the swept contact; default is False

frequency: (list of numbers) list of frequencies used when sweeping the capacitance, default is [1.0,]

VolumeIntegrateList: (list) list of models for which volume integration is required, empty by default

DeviceMonitorList: (list) list of models to be monitored, empty by default

SaveAll: (Boolean) whether to save the device data at every step, default is False

SaveData: (Boolean) whether to save the data specified by SweepModel during the sweep, default is True

SaveFinal: (Boolean) whether to save the device data of the last step, default is True

SaveZero: (Boolean) whether to save the device data when the sweep voltage is 0 V, default is False

SaveCurrentVariation: (numeric) minimum change of the current when saving the device, 0

SaveRange: (list) save the device file only within the specified voltage range; the default is all, [0, 0]

PlotResults: (Boolean) whether to display in real time, default is True

ExtraNodeChargeUpdate: (Boolean) whether to update the extra charge, default is True

```
PlotTransient(device,
    VoltateContacts = [],
    CurrentContacts = [],
    ChargeContacts = [],
    CapacitorContacts= [],
    TimeModel      = "Log",
    minT_Step      = 1e-15,
    maxTime        = 1e-5,
    rel_error       = 1e-8,
    abs_error       = 1e30,
    iterations      = 30,
    FerroRegion     = None,
    ElementChecklist= None,
    Lable           = "",
    SaveAll          = False,
    SaveData         = True,
    SaveFinal        = True,
    SaveCurrentVariation=0,
    SaveRange        = [0,0],
    PlotResults      = True,
    NodeChecklist =[]
)
```

Calculate the transient variation, display the device variation, and print and save the results

Parameters:

device: (string) device name

VoltateContacts: (list of strings) contacts whose voltage needs to be monitored,  
default is []

CurrentContacts: (list of strings) contacts whose current needs to be monitored,  
default is []

ChargeContacts: (list of strings) contacts whose charge needs to be monitored,  
default is []

CapacitorContacts: (list of strings) contacts whose capacitance needs to be  
monitored, default is []

TimeModel: (string) time variation mode, default is "Log"

minT\_Step: (numeric) minimum time interval, default is 1e-15

maxTime: (numeric) 1e-5

rel\_error: (numeric) tolerated relative error, default is 1e-8

abs\_error: (numeric) tolerated absolute error, default is 1e30

iterations: (numeric) tolerated number of iterations, default is 30

FerroRegion: (string) ferroelectric region name, default is None

ElementChecklist: (list of strings) element edge models to be monitored, default is  
None

Lable: (string) label of the saved file, default is "

SaveAll: (Boolean) whether to save at every step, default is False

SaveData: (Boolean) whether to save the data, default is True

SaveFinal: (Boolean) whether to save the final result, default is True

SaveCurrentVariation: (string) minimum interval for saving the current variation,  
default is 0

SaveRange: (string) voltage range for saving the file, [0, 0]

PlotResults: (string) whether to display the calculation results in real time, default  
is True

NodeChecklist: (list of strings) node models to be monitored, default is None []

## 7) [QSdiode\\_common.py](#)

Create one-dimensional and two-dimensional diode simulation devices

CreateMesh(device, region, Length, refinescale):

Create a one-dimensional diode structure

Parameters:

device: (string) device name

region: (string) region name

Length: (numeric) device length

refinescale: (numeric) minimum interval

CreateMesh2(device, region):

Create the default one-dimensional diode structure

Parameters:

device: (string) device name

region: (string) region name

Create2DMesh(device, region):

Create the default two-dimensional diode structure

Parameters:

device: (string) device name

region: (string) region name

Create2DGmshMesh(device, region):

Create a two-dimensional diode structure from the Gmsh file gmsh\_diode2d.msh

Parameters:

device: (string) device name

region: (string) region name

Create3DGmshMesh(device, region):

Create a three-dimensional diode structure from the Gmsh file gmsh\_diode3d.msh

Parameters:

device: (string) device name

region: (string) region name

SetNetDoping(device, region, DonorDoping=1e18, AcceptorDoping=1e18, Length=1e-5):

Set the basic device parameters

Parameters:

device: (string) device name

region: (string) region name

DonorDoping: (numeric) donor doping concentration, default is 1e18

AcceptorDoping: (numeric) acceptor doping concentration, default is 1e18

Length: (numeric) position of the metal junction, default is 1e-5

InitialSolution(device, region, circuit\_contacts=None):

Calculate the initial concentrations

Parameters:

device: (string) device name

region: (string) region name

circuit\_contacts: (list of strings) names of the contacts connected to the circuit

DriftDiffusionInitialSolution(device, region, circuit\_contacts=None)

Create the electron and hole continuity equations

Parameters:

device: (string) device name

region: (string) region name

circuit\_contacts: (list of strings) names of the contacts connected to the circuit

## 5. Examples

This chapter uses a ferroelectric thin-film transistor as an example to illustrate the detailed usage steps of this software.

The ferroelectric transistor involves four files in total:

1) [mos\\_BotContact.geo](#)

The source file for creating the mesh file with Gmsh; for detailed usage, please refer to the Gmsh user manual

2) [mos\\_BotContact-intLay2.msh](#)

The created mesh file, which is imported by the third Python file to create the device structure

3) [create\\_mos.py](#), the Python file for creating the device structure

```
# Import modules
from devsim import *
sys.path.append('[storage path]')
sys.path.append('../')
from QSimple_physics import *

# Set the device name
device="mos_BotContact"

# Import the Gmsh file and create the device
create_gmsh_mesh(file='mos_BotContact-intLay2.msh', mesh=device)
# Set the device structure parameters
device_width    = 1.0 #um
gate_width      = 0.8 #um
bulk_thickness   = 4e-2 #um
air_thickness    = 4e-1 #um
oxide_thickness  = 2e-1 #um
diffusion_thickness = 5e-3 #um

x_diffusion_decay = 5e-3 #um
y_diffusion_decay = 5e-3 #um
refine_spacing    = 1e-2 #um

x_bulk_left = 0.0
x_bulk_right = x_bulk_left + device_width
x_center = 0.5 * (x_bulk_left + x_bulk_right)
x_gate_left = x_center - 0.5 * (gate_width)+x_diffusion_decay
x_gate_right = x_center + 0.5 * (gate_width)-x_diffusion_decay
x_device_left = x_bulk_left - air_thickness
```



```

x_device_right = x_bulk_right + air_thickness

y_bulk_top = 0.0
y_oxide_top = y_bulk_top - oxide_thickness
y_oxide_mid = 0.5 * (y_oxide_top + y_bulk_top)
y_bulk_bottom = y_bulk_top + bulk_thickness
y_bulk_mid = 0.5 * (y_bulk_top + y_bulk_bottom)
y_device_bottom = y_bulk_bottom + air_thickness
y_air_top = y_bulk_bottom + air_thickness
y_diffusion = y_bulk_top + diffusion_thickness

# Name the Gmsh regions
add_gmsh_region (mesh=device, gmsh_name="bulk", region="bulk",
material="Silicon")
add_gmsh_region (mesh=device, gmsh_name="oxide", region="oxide",
material="Oxide" )
add_gmsh_region (mesh=device, gmsh_name="air", region="air", material="Air")
add_gmsh_contact (mesh=device, gmsh_name="drain_contact", region="bulk",
name="drain", material="metal")
add_gmsh_contact (mesh=device, gmsh_name="source_contact", region="bulk",
name="source", material="metal")
add_gmsh_contact (mesh=device, gmsh_name="body_contact", region="air",
name="ground", material="metal")
# Name the contacts
add_gmsh_contact (mesh=device, gmsh_name="gate_contact", region="oxide",
name="gate", material="metal")
add_gmsh_contact (mesh=device, gmsh_name="attached_source_contact",
region="oxide", name="attached_source", material="metal")
add_gmsh_contact (mesh=device, gmsh_name="attached_drain_contact",
region="oxide", name="attached_drain", material="metal")
# Name the interfaces

add_gmsh_interface (mesh=device, gmsh_name="bulk_air_interface",
region0="bulk", region1="air", name="bulk_air")
add_gmsh_interface (mesh=device, gmsh_name="bulk_oxide_interface",
region0="bulk", region1="oxide", name="bulk_oxide")

# Finish importing the Gmsh file
finalize_mesh(mesh=device)

# Create the device
create_device(mesh=device, device=device)

# Doping concentration parameters

```

```

bulk_doping = 1e3 #1um^3
body_doping = 1e7 #1um^3
#n doping
drain_doping = -1e7 #1um^3
source_doping = -1e7 #1um^3
gate_doping = -1e8 #1um^3
# Set the device doping
SetDopingParameters(device, "bulk",
                    bulk_doping =bulk_doping,
                    body_doping =body_doping,
                    drain_doping =drain_doping,
                    source_doping=source_doping,)

# Set the device structure parameters
mydict = {}
mydict["drain_doping"] = drain_doping
mydict["body_doping"] = body_doping
mydict["gate_doping"] = gate_doping
mydict["source_doping"] = source_doping
mydict["bulk_doping"] = bulk_doping
mydict["x_gate_left"] = x_gate_left
mydict["x_gate_right"] = x_gate_right
mydict["x_diffusion_decay"] = x_diffusion_decay
mydict["y_diffusion"] = y_diffusion
mydict["y_bulk_bottom"] = y_bulk_bottom
mydict["y_diffusion_decay"] = y_diffusion_decay

# Set the doping concentration model
node_model(name="DrainDoping", device=device, region="bulk",
equation="0.25*%(drain_doping)1.15e*erfc((x-%(x_gate_left)1.15e)/%(x_diffusion_de
cay)1.15e)*erfc(-(y_diffusion)1.15e-y)/%(y_diffusion_decay)1.15e)" % mydict)
node_model(name="SourceDoping", device=device, region="bulk",
equation="0.25*%(source_doping)1.15e*erfc(-
(x-%(x_gate_right)1.15e)/%(x_diffusion_decay)1.15e)*erfc(-(y_diffusion)1.15e-
y)/%(y_diffusion_decay)1.15e)" % mydict)
node_model(name="NetDoping", device=device, region="bulk",
equation="DrainDoping + SourceDoping +%(bulk_doping)1.15e + 1" % mydict)

```

4) ferro\_BottomContact.py, the file for setting the device parameters and simulating the device voltage sweep

```
# Import the required modules

from devsim import *
import sys, devsim
import os, tarfile
sys.path.append('[storage path]')
sys.path.append('../')
from QSimple_physics import *
from QSPlotSweep import *
from QSFerro import *
from QSramp import *


# Set the device to run with 128-bit extended precision
SetExtendedPrecision()
SetMicroMeterBasicParameters()
# Run the Python file that creates the device

import create_mos
# Device structure name
device      = "mos_BotContact"
silicon_regions = ("bulk",)
oxide_region  = "oxide"
air_region    = "air"
oxide_regions = ("oxide",)
regions      = ("bulk", "oxide", air_region)
interfaces    = ("bulk_oxide", "bulk_air")
FerroRegion   = oxide_region
# Create the potential and electric field variables
for i in regions:
    CreatePotentialAndFlux(device, i)

# Set the physical parameters and models of the silicon region
mu=1e5
for i in silicon_regions:
    SetMicroMeterSiliconParameters(device, i, 300, pMobility=mu, nMobility=mu)
    CreateSiliconPotentialOnly(device, i)

# Set the physical parameters and models of the oxide region
for (c,r,k) in (("ground",air_region,1.0),
                ("gate",oxide_region,3.9)):
    SetOxideParameters(device, r, rPermittivity=k)
```

```

CreateOxidePotentialOnly(device, r, "log_damp")
CreateOxidePotentialContact(device, c)

# Set the physical parameters and models of the contacts in the silicon region
contacts=("source", "drain" )
for c in contacts:
    tmp = get_region_list(device=device, contact=c)
    r = tmp[0]
    print("%s %s" % (r, c))
    SetContactDopingOffset(device, c)
    CreateSiliconPotentialOnlyContact(device, c)

# Set the physical parameters and models of the contacts in the oxide region
attached_contacts=(("attached_source","source"), ("attached_drain", "drain"))
for (i,j) in attached_contacts:
    SetContactPotentialOffset(device, j, GetContactDopingOffset(device, j))
    CreateOxidePotentialContact(device, i, attached_to=j, offset_contact=j)

# Set the interfaces
for i in interfaces:
    CreateSiliconOxideInterface(device, i)

# Calculate the initial values of the device
InitialSolve(device,rel_error=1e-9)

# Create the electron and hole solution variables and continuity equations in the
silicon region
for i in silicon_regions:
    CreateElectronAndHoleSolutions(device, i)
    CreateSiliconDriftDiffusion(device, i)

# Create the contact boundary conditions of the electron and hole continuity
equations in the silicon region
for c in ("source", "drain"):
    tmp = get_region_list(device=device, contact=c)
    r = tmp[0]
    CreateSiliconDriftDiffusionAtContact(device, c)

# Set the saturation mobility model

```

```

V_Sat=1.0e11
for r in ("bulk",):
    DeleteEdgeModelAndDerivatives(device, r, "HoleCurrent")
    ElectricFieldMagnitude(device, r)
    MicorSaturationMobilityParameters(device, r,V_n_Sat=V_Sat, V_p_Sat=V_Sat)
    QSSaturationMobility(device, r, "Hole")
    # LorentzSaturationMobility(device, r, "mu_n_Sat", "mu_n_eff", "LorentzEField")
    ElementHoleCurrent2d(device, r)
    ElementHoleContinuityEquation(device, r)

# Set the contact boundary conditions of the electron and hole continuity equations
with element edge models
for contact in ( "drain", "source"):
    ElementContactHoleContinuityEquation(device, contact)

# Calculate the initial values after the model is changed
InitialSolve(device, rel_error=1e-9)

# Set the model of the ferroelectric region
if FerroRegion!=None:
    print("*****Creat Ferro and Solve")
    MicroMeterFerroParameters(device, FerroRegion)
    CreateFerroRegion(device, FerroRegion, update_type="log_damp")
    CreateOxidePotentialContact(device,"gate", element_contact=True)
    for (i,j) in attached_contacts:
        SetContactPotentiOffset(device, i, GetContactDopingOffset(device, j))
        CreateOxidePotentialContact(device, i, attached_to=j, offset_contact=j,
element_contact=True)

# Gradually adjust the saturation polarization of the ferroelectric region to calculate
the initial values
    SpecificValueInitiateRamp(device, FerroRegion, "SaturationPolarization",
abs_error=1e30, rel_error=1e-9, iterations=30)

# Set the model of the absolute value of the electric field vector
for r in regions:
    CreateAbsCurrentAndEfield(device, r, magnitude=True)

# Set the models to be monitored
ElementChecklist=("#("SweepDirectionY","CoerciveSignY","OldPCoefficientY","NewPC
oefficientY",

```

```

"NumeratorTanhY","DenominatorTanhY","PolarizationY","PreElectricField_y","Electric
Field_y")
# Set the voltage sweep range
Coersive_bias = 10.00
Other_bias   = -5
#Star_bias   = max(Other_bias,0)+3*Coersive_bias
Star_bias    = min(Other_bias,0)-3*Coersive_bias
End_bias     = 16 #ther_bias,0)-3*Coersive_bias
# Set the device label, which is used as the folder name

Description="1e-16V_Sat%0.2e_mu%0.2e_D%sG%s~%s"%(V_Sat, mu, Other_bias,
Star_bias, End_bias )
set_parameter(device=device, name="Description", value=Description)


# Save the initial state file
QSSaveDevice(device, file="InitialSolve.tec",ftype="tecplot")


# Set the voltage sweep speed
set_parameter(device=device, name="SweepSpeed", value=10)


# Set the contacts whose charge needs to be calculated
ChargeContacts=["attached_source","attached_drain","gate"]


# Set the sweep step information
SweepSettings=( ("drain", Other_bias, "pre", 0.5, False, False),
                ("gate", Star_bias, "star", 0.5, False, True ),
                ("gate", End_bias, "forw", 0.1, True, True ),
                ("gate", Star_bias, "back", 0.1, True, True ),
                )


# Start the voltage sweep
for (SweepContact, Bias, Lable, step_limit, SaveZero, SaveFinal) in SweepSettings:
    Msg=VoltagePlotSweep(device, SweepContact=SweepContact, CurrentContacts =
["drain"],End_bias=Bias, FerroRegion=FerroRegion, ChargeContacts=ChargeContacts,
iterations=30, step_limit=step_limit, min_step = 0.005, rel_error=1e-18, Lable=Lable,
ElementChecklist=ElementChecklist, SaveAll=False, PlotResults = True)

```